Stackless vs. Stackful Coroutines: A Comparative Study for RDMA-based Asynchronous Many-Task (AMT) Runtimes

Mia Reitz and Jonas Posner

jonas.posner@uni-kassel.de
www.jonasposner.com

UNIKASSEL VERSITÄT

Hochschule Fulda
University of Applied Sciences

16th November 2025

Motivation

- Modern supercomputers:
  - Increasing complexity ⇒ programmability challenges for MPI+X
- **Asynchronous Many-Task (AMT)**
  - Promising abstraction; especially for irregular, dynamic workloads
  - Load balancing via work stealing
    - **One key challenge**: suspend and migrate tasks efficiently across cluster nodes
    - The efficiency of suspending, migrating, and resuming is critical to application performance

⇒ **Our Goal:** Compare *stackful* vs. *stackless* coroutine implementations in Remote Direct Memory Access (RDMA)-based AMT runtimes

Contributions

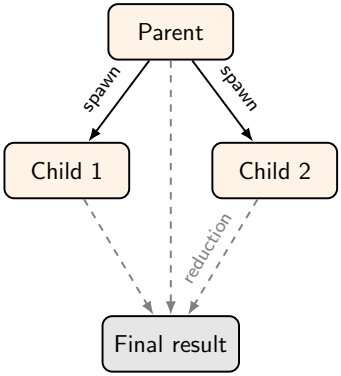
- Mechanism for migrating **C++20 stackless coroutines** across processes (cluster-nodes) via RDMA
- Two RDMA-based AMT runtimes (*prototypical*):
  - Stackful (uni-address scheme)
  - Stackless (migratable C++20)
- Experimental comparison of:
  - Task creation
  - Context switching
  - Communication overhead

Background: AMT

- **Asynchronous Many-Task (AMT)**
  - Programmers split the computation into a large number of fine-grained tasks
  - Tasks are dynamically mapped to workers (e.g. processes distributed across cluster nodes)
- **Task Models**
  - Dynamic Independent Tasks (DIT)
  - Nested Fork-Join (NFJ)

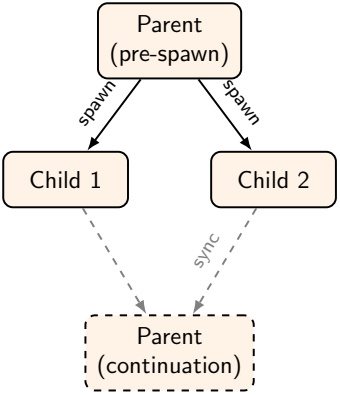
Background: Task Models

Dynamic Independent Tasks (DIT) using child-stealing



Parent tasks complete after spawning. All task results are combined into the final result by reduction.

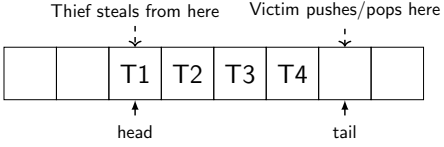
Nested Fork-Join (NFJ) using continuation-stealing



Parent continuation is suspended until children complete. Children return their results to their parent.

Work Stealing

- Dynamic load balancing: idle workers (*thieves*) steal from *victims*
- RDMA-based stealing bypasses the victim CPU and directly access victim's memory
- We use **dynamic independent tasks** and **coordinated random work stealing** with **continuation-stealing**
  - Execute most recently spawned task locally → good cache locality
  - Parent's remaining work (*continuation*) becomes stealable
  - Requires **suspendable** tasks for migration and resume



Stackful vs. Stackless Coroutines

- Suspendable tasks in C++ AMT runtimes are typically implemented as coroutines

Stackful

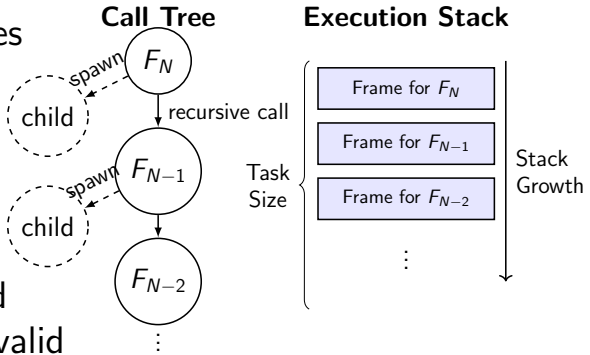
- Suspend anywhere (flexible)
- Fast task creation
- Large migration payload
- Higher context switch cost

Stackless

- Minimal, constant payload
- Low context switch cost
- Slower creation (heap alloc)
- Restricted suspend points

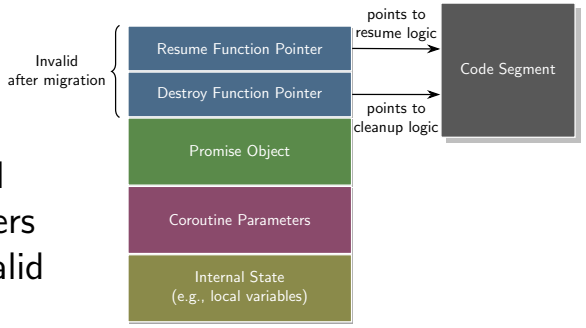
Stackful Implementation (*traditional*)

- Based on **uni-address scheme** (*Shiina & Taura, 2022*)
  - Uni-address region: identical virtual address on all processes
    - Execution stack for tasks
- Migration:
  - Thief performs `MPI_Get()` on victim's stack region
  - Suspended parent task copied to same address ⇒ pointers valid
  - Resume via fast assembly-level context switch



Stackless Implementation (*C++20*)

- **No** own stack
- Compiler transforms a coroutine into a state machine object
- Each coroutine = heap-allocated frame object with internal pointers
- **Problem:** function pointers invalid after migration
- **Solution: Pointer patching**
  - Exchange base addresses between processes
  - Adjust internal function pointers after transfer
  - **Enables first migration of C++20 stackless coroutine**



Experimental Setup

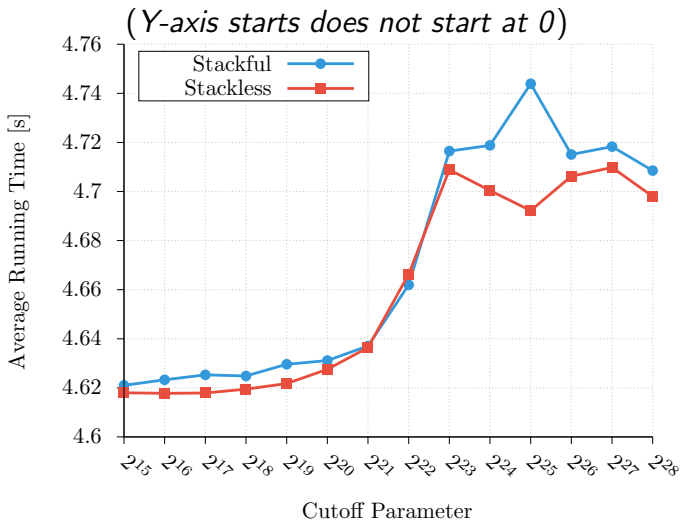
- 2 nodes, $2 \times$ AMD EPYC 7282 (64 cores total), Infiniband RDMA
- OpenMPI 5.0.2, g++ 15.1.1
- Synthetic Benchmark: recursive Monte Carlo π
  - Mix of DIT + continuation-stealing (NFJ)
    - Allows isolating and measuring performance without dependency tracking
  - $N = 2^{34}$ points; cutoff C controls task granularity
- Metrics: total running time, task size, creation, switch, steal duration

Fixed-Granularity Results ($C = 2^{16}$)

- Fine-grained tasks: duration of $\approx 17\,\mu s$
- $\sim$Equal total running time (≈ 4.62 s)
- Creation: *stackful* $2.4\times$ faster
- Switch: *stackless* $3.5\times$ faster
- Migration dominated by network latency, not task size

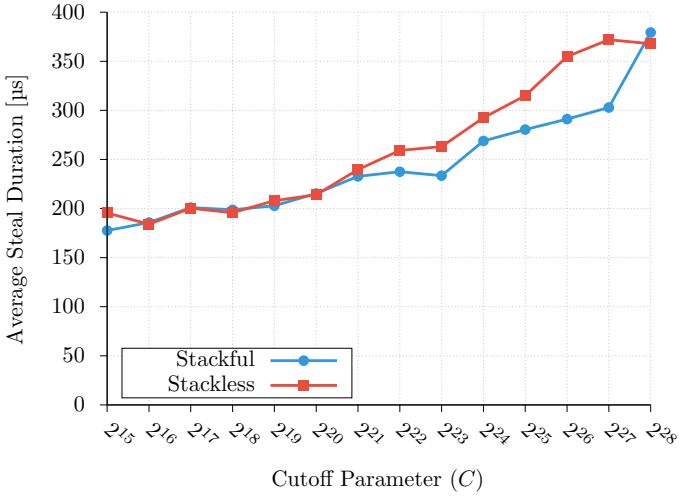
Metric	Stackful Coroutines	Stackless Coroutines
Running Time	$\sim$4.621 s	$\sim$4.618 s
Task Size	279 bytes	112 bytes
Task Creation	$\sim$40 ns	$\sim$98 ns
Context Switch	$\sim$109 ns	$\sim$32 ns
Steal Duration	$\sim$195,000 ns	$\sim$193,000 ns

Variable Granularity: Total Running Time



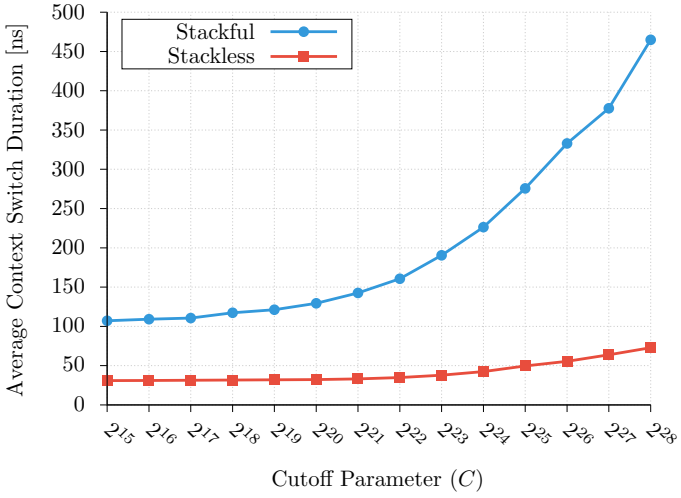
- **Total running time:** Nearly identical between stackful and stackless
- Higher cutoff: Better app locality, worse AMT runtime locality
- Long compute phases lead to cache-miss penalties for runtime operations

Variable Granularity: Average Steal Duration



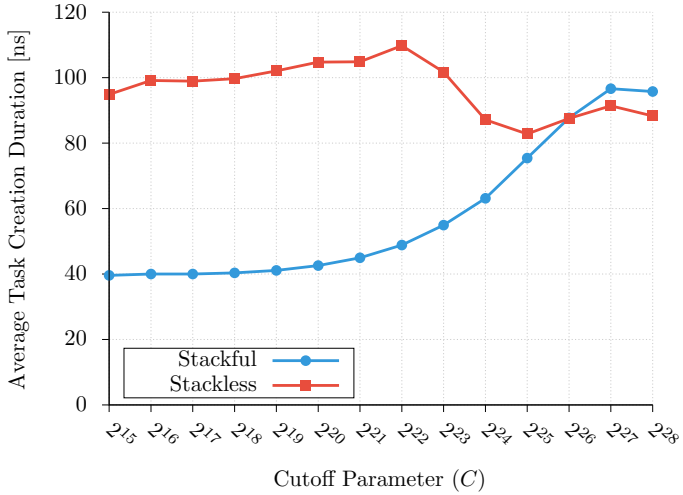
- **Steal duration:** Increases with increasing cutoff

Var. Gran.: Average Context Switch Duration



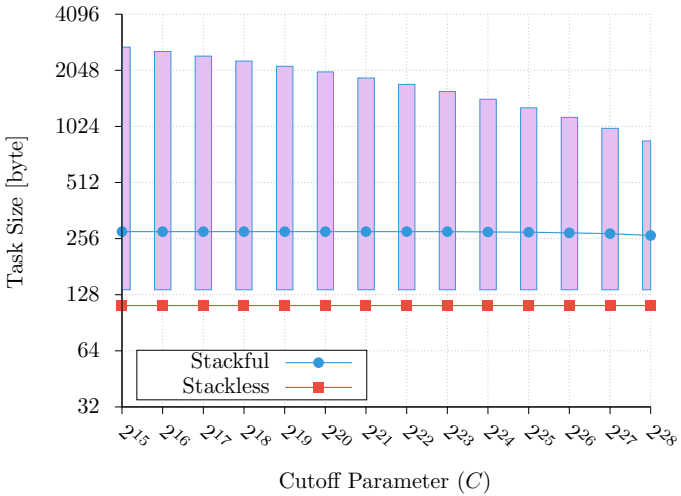
- **Context Switch Duration:** Increases with increasing cutoff
- Stackful slows down by $\sim 4\times$
- Stackless only $\sim 2\times$ slower
- Cache misses dominate after long compute phases

Variable Granularity: Average Task Creation



- Stackful creation gets slower with increasing cutoff
- Stackless stays mostly "constant"
  - Largely immune to cache effects

Variable Granularity: Average Task Size



- Boxes show the range of task sizes
- Lines show the average task size
- Size of a stackful continuation is proportional to its recursion depth
- Stackless has constant size

Conclusion and Outlook

- Compared stackful and stackless coroutines in RDMA AMT runtimes
- Both show nearly identical performance
  - Stackful: faster creation (up to 2.4×)
  - Stackless: faster switching (up to 3.5×); smaller task states
  - Smaller migration task size *does not* reduce communication time → latency-dominated
  - Stackless gains relevance as task state grows
- **Choice depends on task granularity!**
- Future work:
  - Experiments with benchmarks having larger task states
  - Hybrid AMT runtime that adaptively selects the coroutine type

Thank you for your attention!

Questions?

jonas.posner@uni-kassel.de

www.jonasposner.com

github.com/projectwagomu

zenodo.org/communities/projectwagomu